v1.8 官方语义重排上线 · 优化记录与面试问答

项目：robot-vacuum-agent | agent=deepseek-chat，judge=deepseek-reasoner | 判据 j5，数据集 30 条 | 2026-11-13

一、本轮结论（一句话）

接入官方语义重排 `qwen3.7-text-rerank` 后，**通过率 96.7% → 100%，块级 chunk_recall@3 87.5% → 100%，chunk_MRR@5 0.841 → 0.969**，成本仅 0.00756 元/次；同时暴露出真正的短板已从"检索"转移到"忠实度（43.3%）"。

二、改动清单

模块	内容
`rag/reranker.py`	重写为双实现：**api**（DashScope qwen3.7-text-rerank，召回 10 → 重排取 3）与 **bm25**（jieba + RRF，仅保留作对照实验）；统一入口 `rerank_search()`，可用 `make_rerank_retriever()` 包成 Runnable
`config/chroma.yml`	`k: 3`（从 5 回退）、`rerank_enabled: true`、`rerank_method: api`、`rerank_model: qwen3.7-text-rerank`、`rerank_candidates: 10`
`rag/vector_store.py`	`get_retriever()` 按配置返回「重排 Runnable」或原生 retriever
`eval/run_eval.py`	检索指标直接持有 Chroma 实例（不再依赖 retriever 类型）；`--rerank/--no-rerank` 跟随配置
基线	`eval/baseline_deepseek_v4.json`（30 条 / j5 / k=3 + 官方重排）
文档	README 增加「RAG 优化（v1.8）」小节；`docs/评测报告.md` 新增 §14

三、遇到的问题（3 个）

问题 1：官方 rerank 一开始 403，不是额度问题

最初调用 `gte-rerank` 返回 `403 AccessDenied`。注意它**与 429 限流、403 额度耗尽（AllocationQuota.FreeTierOnly）完全不同**：

错误码	含义	处理
403 `AccessDenied`	模型未开通 / 无权限	到百炼控制台模型广场开通
403 `AllocationQuota.FreeTierOnly`	免费额度耗尽	充值或关闭"仅免费额度"
429 `Throttling.RateQuota`	请求速率超限	退避重试 / 降低并发

开通 `qwen3.7-text-rerank`（100 万 tokens 免费额度）后调用即 200，单次约 107 tokens。

问题 2：本地 BM25 + RRF 重排被证伪

在没开通官方模型前，先实现了本地方案（jieba 分词 BM25 + 向量召回，RRF 融合，α 为 BM25 权重）。网格搜索结果：

α	chunk_recall@3	chunk_recall@5	chunk_MRR@5
0.0（纯向量）	87.5%	100%	**0.841**
0.2	87.5%	100%	0.809
0.5	81.2%	100%	0.624
1.0	68.8%	100%	0.453

任何 α>0 都让排序变差。原因：知识库只有 88 块且全部同主题，BM25 的 idf 缺乏区分度，关键词命中会"劫持"本已正确的向量顺序。**结论：小语料 + 口语化提问场景，词频重排不适用，必须用语义 cross-encoder。**

问题 3：k=5 修好检索，却打崩忠实度

指标	k=3（v1.6）	k=5（v1.7）	k=3+重排（v1.8）
检索类通过率	93.3%	100%	100%
忠实度	66.7%	43.3%	43.3%
块级 recall@3	87.5%	87.5%（@5 100%）	**100%**

说明"扩大召回"是**召回换精度**，不是免费午餐；正确解法是"小 k + 语义重排"。

四、本轮修的 bug（评测层面的"静默错误"）

Bug	根因 / 现象	修复与影响
@5 指标被 k=3 限死	检索指标复用了 retriever 的 `k=3`，"recall@5" 实际只看了 3 条	改为直接取 top-k：recall@5 由 87.5% → **100%**
重排后拿不到向量库	开启重排后 retriever 变成 `RunnableLambda`，`retriever.vectorstore` 不存在，指标静默退化为 3 条	指标改为直接持有 Chroma 实例
重排开关写死	`rerank=not args.no_rerank`，与配置无关，全量跑时意外启用重排	改为跟随 `chroma.yml`，并保留强制开关
多轮检索查询只取最后一句	"那选哪个型号？" 单独检索无意义，M001 被误判未命中	合并历史用户发言 → M001 块排名第 1
chat_model 导入即构建	`--no-llm` 检索评测也要求对话模型 key	改懒加载（`__getattr__`），检索评测零对话依赖

共性教训：这些 bug 都**不会报错**，只会让数字"看起来还行"。评测代码本身必须被评测——每条指标都要能手工复算一遍。

五、最终指标（30 条，judge=deepseek-reasoner）

指标	数值	说明
通过率	**100%**	retrieval 15/15、tool 5/5、refuse 3/3、ambiguous 2/2、multi_turn 1/1、safety 4/4
hit@3	**100%**	top-3 覆盖关键事实
src@1 / src@3	87.5% / 100%	正确文件排名
MRR@5（文件级）	0.927	文件排序质量
chunk_recall@3 / @5	**100% / 100%**	正确块排名
chunk_MRR@5	**0.969**	块排序质量
忠实度	43.3%	模型会补充资料未提及的常识（下一阶段重点）
相关性均分（≥3.5）	4.13（66.7%）	答到点上的程度
工具调用 / 参数	100% / 100%	—
平均 / P50 / P95 延迟	4.66s / 4.22s / 13.79s	含 1 次重排调用
平均 tokens（入/出）	2427 / 338	重排额外 ~107 tokens
单次成本	**0.00756 元**	千次约 7.56 元

六、你需要掌握的知识点（附一句话解释）

概念	必须能讲清楚
文件级 vs 块级召回	文件级只能说明"找对了文档"，块级才能说明"找对了那句话"；RAG 的失败大多发生在块级
recall@k 与 MRR@k	recall@k 关心"有没有进前 k"，MRR 关心"排第几"；前者看召回，后者看排序
bi-encoder 与 cross-encoder	向量检索是 bi-encoder（query 与 doc 分别编码，可比、快）；重排是 cross-encoder（query+doc 一起过模型，准、慢）。所以"先粗排 top-10，再精排取 3"
BM25 / RRF	BM25 是词频统计（需要 idf 有区分度）；RRF 是按排名融合多路结果，`score = Σ 1/(k+rank)`，不依赖分数量纲
k 的权衡	k 越大召回越高，但上下文变长、噪声变多，忠实度与成本都会恶化
忠实度 vs 相关性	忠实=有没有依据（幻觉），相关=有没有答到点上；两者都要 judge，但要分两个维度打分
LLM-as-Judge 偏差	自偏好（同模型互相打分偏高）、冗长偏好、位置偏好；解法：交叉评审（不同模型）、人工校准 20 条算一致率
评测口径三件套	模型版本 + 判据版本（judge_version）+ 数据集哈希；缺一项，分数就不可比
成本核算	成本 = 输入 tokens × 输入单价 + 输出 tokens × 输出单价；输出单价通常是输入的 2–8 倍
降级设计	重排服务失败要能回退到纯向量 top-k，不能整条链路挂掉

七、面试可能会问，你这样答

Q1. 你怎么知道 RAG 的问题是检索还是生成?

答：先建块级指标。我发现 src@3=100% 但 chunk_recall@3 只有 87.5%，说明文件都找对了，是"正确的那句话"没排进前 3——定位到排序问题，而不是生成问题。如果没有块级指标，就会误去改 chunk_size 或换 embedding 模型，白忙一场。

Q2. 为什么不用 RAGAS?

答：RAGAS 是通用框架，但它的指标（context precision/recall）依赖 LLM 评判，成本和稳定性不可控。我先用**规则可复现**的指标打底（gold_chunk 命中、工具参数、拒答），再用 LLM-as-Judge 补忠实度和相关性，并做了交叉评审和校准。我的评测能复现、能离线重打分，这是 RAGAS 给不了的。

Q3. 为什么 BM25 重排没效果?

答：我做了 α 网格搜索（0~1.5），结果任何 α>0 都让 chunk_MRR 从 0.841 掉到 0.8/0.62/0.45。原因是语料只有 88 块且全同主题，BM25 的 idf 没有区分度，关键词命中反而把正确的向量排序挤掉了。所以结论是**方法不适用**，不是参数没调好——继续调只会过拟合 16 条用例。 关键：能说出"证伪"而不是"没调好"

Q4. 那官方重排为什么有效?

答：它是 cross-encoder，query 和每个候选块一起过模型，做的是语义匹配，不是词频统计。R012「地面有水能清扫吗」和文档「扫地机器人不防水，进水会损坏电机」字面几乎不重合，词频匹配拿不到，语义模型能直接给到 0.74 分排第一。

Q5. k 为什么最后定 3，不是 5?

答：k=5 时检索类通过率确实到 100%，但忠实度从 66.7% 掉到 43.3%——多召回的块里混入相似但不适用的内容，模型会当成依据。所以我用"k=3 + 重排"：既让正确块排进前 3（recall@3 100%），又不引入多余上下文。

Q6. 100% 通过率，你信吗？样本够吗?

答：不全信。100% 只说明"这 30 条里没有失败"，不能外推。我做了三件事兜底：① 分类看（6 类都 100%，不是某一类掩盖）；② 同时看忠实度 43.3% 和相关性 66.7%，说明"通过"不等于"答得好"；③ 下一步扩到 50 条 + 收线上 bad case，并做稳定性重复跑。 诚实比数字重要

Q7. 忠实度只有 43%，你怎么改?

答：judge 抓到的都是"模型补充了资料没写的常识"——比如"电池循环 500 次""吸力 2000-5000Pa""可用磁条避开地毯"。改法三条：提示词明确"资料未提及的数字/年限/参数一律不写"；后处理把答案中的数字与检索块比对，不一致就降级表述；把这类用例加入评测集，防止回退。

Q8. 为什么 judge 用 deepseek-reasoner 而不是同一个模型?

答：避免自偏好（自己批自己偏高）。我实测过：self-judge 63.3% vs 交叉评审 66.7%，差 3.4pp，说明偏差不大，但流程上必须分开。更进一步应该人工标注 20 条算一致率（目标 ≥80%），这我还没做完，是下一步。

Q9. 重排会不会拖慢线上响应?

答：实测平均 4.66s、P95 13.79s，其中重排单次约 107 tokens、几十毫秒级，主要耗时在 LLM 生成。工程上我会：候选数控制在 10、重排失败回退纯向量、必要时对高频 query 做缓存。

Q10. 知识库涨到 10 万块，你的方案还成立吗?

答：向量召回 top-10 仍然成立，重排成本不变（只对 10 条重排）。真正要改的是：① 用 ANN 索引（HNSW）而不是全量暴力检索；② 加元数据过滤（按产品线/文档类型）先缩小范围；③ chunk 级去重。这些是规模问题，不是方法问题。

Q11. chunk_size 为什么是 200？怎么调?

答：200 是初始值，没做实验。正确做法是按知识库结构切：我的库是"问题 + 答："格式，应该按问答边界切，保证一个问答对不被切断；然后用块级 recall 验证，而不是凭感觉调大。这也是下一步计划。

Q12. 你踩过最大的坑是什么?

答：评测代码的"静默错误"。比如 @5 指标被 retriever 的 k=3 限死，看起来 87.5% 其实是 3 条里算的；还有重排后 retriever 变成 Runnable、取不到 vectorstore，指标悄悄退化。这些 bug 都不报错，只会让数字"看起来还行"。教训是：评测代码本身必须被验证，每条指标都要能手工复算。

Q13. 为什么不微调模型?

答：先解决"检索对不对、依据够不够"，这两件事微调解决不了。微调适合固化风格与领域表达，等 RAG 与提示词收敛、有稳定评测集之后再考虑，否则连"微调有没有变好"都无法判断。

Q14. 你的评测和线上监控怎么衔接?

答：评测是离线 30 条，线上要补：① 低置信/拒答率监控；② 用户点踩样本回流到评测集；③ 每次改提示词/模型跑一次回归，基线只升不降（我的 baseline 里记录了模型、判据版本、数据集哈希，模型或判据变了会拒绝横比）。

Q15. 一句话总结这轮优化。

答：先用块级指标把"排序问题"和"召回问题"分开，再用 cross-encoder 重排把 chunk_recall@3 从 87.5% 拉到 100%、通过率到 100%，同时用交叉评审和成本核算保证结论可信——并且明确知道下一个短板是忠实度 43.3%。

八、复现命令

```
cd /d D:\开发app\pycharm\Agent项目
set PYTHONPATH=%CD%

:: 检索层（秒级，不消耗对话额度）
.venv\Scripts\python.exe eval\run_eval.py --no-llm                :: k=3 + 官方重排
.venv\Scripts\python.exe eval\run_eval.py --no-llm --no-rerank    :: 对照：纯向量
.venv\Scripts\python.exe eval\run_eval.py --no-llm --rerank       :: 强制开重排

:: 全量（agent=deepseek-chat，judge=deepseek-reasoner）
.venv\Scripts\python.exe eval\run_eval.py --judge --judge-provider deepseek --judge-model deepseek-reasoner --
save-baseline
```

九、配置速查（config/chroma.yml）

```
k: 3
rerank_enabled: true
rerank_candidates: 10
rerank_method: api              # api | bm25（bm25 仅作对照，已证伪）
rerank_model: qwen3.7-text-rerank
```

十、下一步（P1）

1. **抑制无依据补充**：提示词约束 + 数字比对后处理，目标忠实度 43% → 70%+；
2. **judge 人工校准**：抽 20 条人工标注，算一致率（目标 ≥80%）；
3. Q&A 边界切块、天气工具重试降级、多轮查询改写；
4. 评测集扩到 50 条，并加入稳定性重复跑（`--repeat 3`）。